Before We Design Our 8-Bit CPU

Reading and watching guide | Schulich on a Chip workshop series

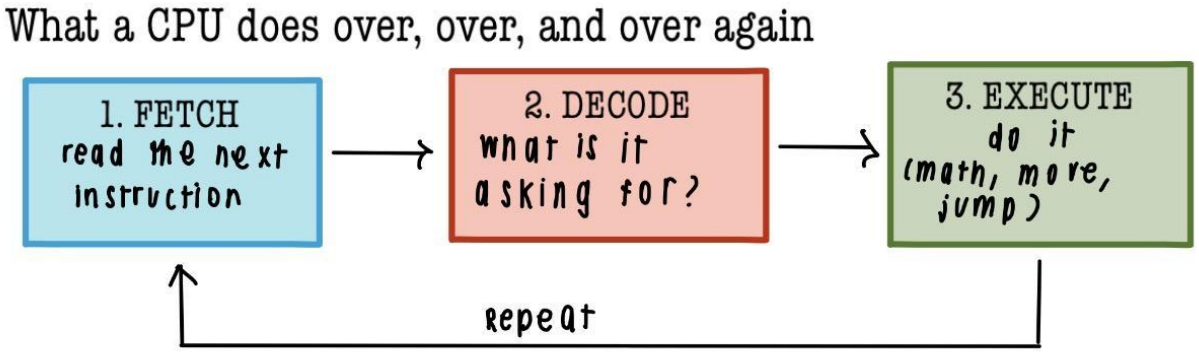
This guide helps you get ready to think about the **RTL** (the Verilog design) of our 8-bit CPU. It answers five questions in simple words:

- What does a CPU do?
- What do we want our CPU to do?
- What parts (modules) does it need?
- How do we decide which instructions it understands (the ISA)?
- What should I watch and read to get ready?

> **How to use this guide**
> 1. Read sections 1 to 4 (about 10 minutes). They are the plan in pictures.
> 2. Watch the videos marked START HERE in section 5.
> 3. Come to the next session with your answers to the "think about" questions in section 9.
>
> You do not need to understand everything. Writing down questions is a great result.

1. What does a CPU do?

A CPU (central processing unit) is the part of a computer that follows instructions. It has a very simple life. It does the same three steps, over and over, billions of times per second:



- **Fetch:** read the next instruction from memory. A counter called the PC (program counter) remembers which one is next.
- **Decode:** work out what the instruction is asking for. Is it "add"? "load a number"? "jump somewhere else"?
- **Execute:** do it. Use the ALU to do math, move a number, or change the PC to jump.

That is all a CPU is: **a machine that reads instructions from memory, one by one, and does what they say.**

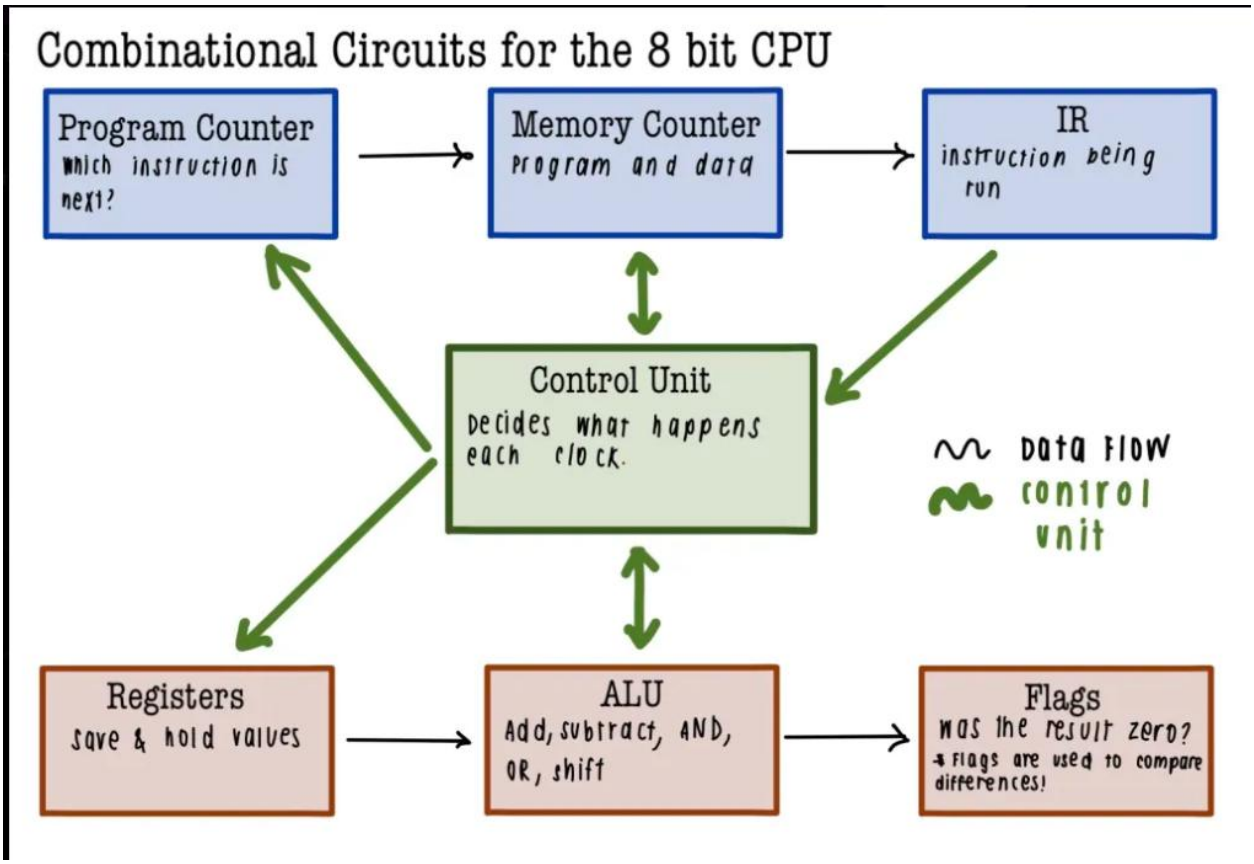
2. What do we want our CPU to do?

This is a decision for our team. Here is a simple target for a first CPU. We will agree on the final choices together:

Goal	What it means
8-bit data	Numbers in registers, in the ALU and in memory are 8 bits wide (0 to 255).
Small memory	For example 16 or 256 bytes. It holds the program and the data.
A few instructions	About 8 to 16, for example load, add, subtract, store, jump and stop.
Runs a tiny program	For example "add two numbers", "count to 10" or "loop forever".
Shows its result	On the Basys 3 board LEDs, with the clock on a button, so we can watch it think.

3. What parts (modules) does it need?

Each box below becomes one Verilog module. We build them one at a time, then wire them together (like Exercise 1 in the Week 0 exercises).



Module	Its job in plain words	Practiced in Week 0
PC	Holds the address of the next instruction. Goes up by 1 each time.	Four-bit counter
Memory	Stores the program (instructions) and the data.	Later
IR	Holds the instruction that is running right now.	Register with byte enable
Registers	Small boxes that hold numbers while we work on them.	Register with byte enable
ALU	Does the math and logic: add, subtract, AND, OR, shift.	Adder-subtractor, shifter
Control unit	A state machine. Each clock it says "now fetch", "now add", "now jump".	Later (an FSM)
Flags	Remember facts about the last result, such as "it was zero". Jumps use them.	Later

4. How do we define the ISA?

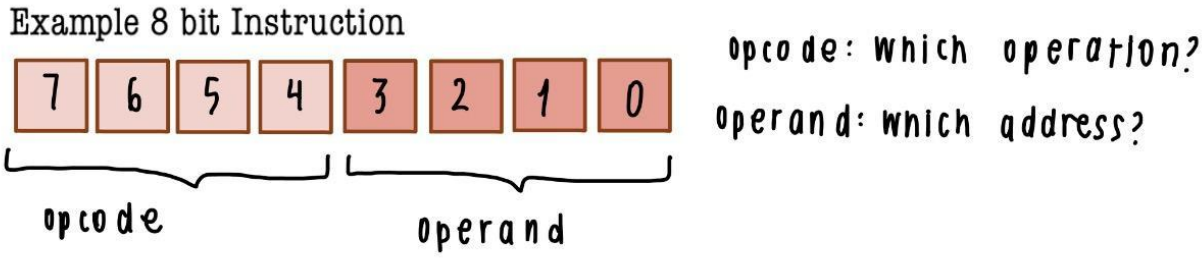
The **ISA** (instruction set architecture) is the list of instructions the CPU understands, and what bits mean each one. Think of it as the CPU's vocabulary. We decide it **before** we write the RTL, because the hardware has to match it.

Six steps to define it

1. **Pick the sizes.** Data width (8 bits), instruction width (for example 8 bits) and memory size.
2. **Pick what storage exists.** One main register (an accumulator), or several registers?
3. **Write programs first.** Write 3 small programs in plain English, such as "load 5, add 3, show the result". The instructions you keep needing become your list.
4. **Group the instructions.** Move data, do math and logic, jump, and stop.
5. **Choose the encoding.** Which bits mean "which operation" (the opcode) and which bits mean "which address" (the operand).
6. **Check the hardware can do it.** For every instruction, ask: can our modules do this in a few clock cycles?

Example only

Here is an example in the style of a classic teaching computer called SAP-1. We may choose something different. The point is to see what an ISA looks like.



Opcode	Name	What it does
0000	LDA	Load a value from memory into the accumulator.
0001	ADD	Add a value from memory to the accumulator.
0010	SUB	Subtract a value from memory from the accumulator.
0011	STA	Store the accumulator into memory.
0100	JMP	Jump: continue from a different address.
0101	JZ	Jump only if the last result was zero.
0110	OUT	Show the accumulator on the LEDs.
0111	HLT	Halt: stop.

5. Videos to watch

Find these on YouTube by searching the **exact title** in the table. Watch the START HERE ones first. For each video, try to answer the question in the last column.

Priority	Video (search this title)	Why watch it	Think about
START HERE	Crash Course Computer Science #7: The Central Processing Unit (CPU)	A short, friendly explanation of the fetch, decode, execute loop.	What are the three steps?
START HERE	Crash Course Computer Science #8: Instructions and Programs	Shows what an instruction looks like as bits.	How is "add" stored as 0s and 1s?
START HERE	Ben Eater: Why build an entire computer on breadboards?	The big picture of a small 8-bit computer, like ours.	Which parts does it have?
Next	Ben Eater: Building the ALU for an 8-Bit Computer	Shows how an ALU adds, and how subtraction uses the adder.	How can an adder also subtract?
Optional	Ben Eater: Programming my 8-bit breadboard computer	Shows a real program running on a tiny CPU.	How does the program change the LEDs?
Optional	Computerphile: Inside the CPU	A deeper look at what is inside a CPU.	Which parts match our diagram?

Ben Eater's full 8-bit computer series is also on his website: eater.net/8bit.

6. Readings

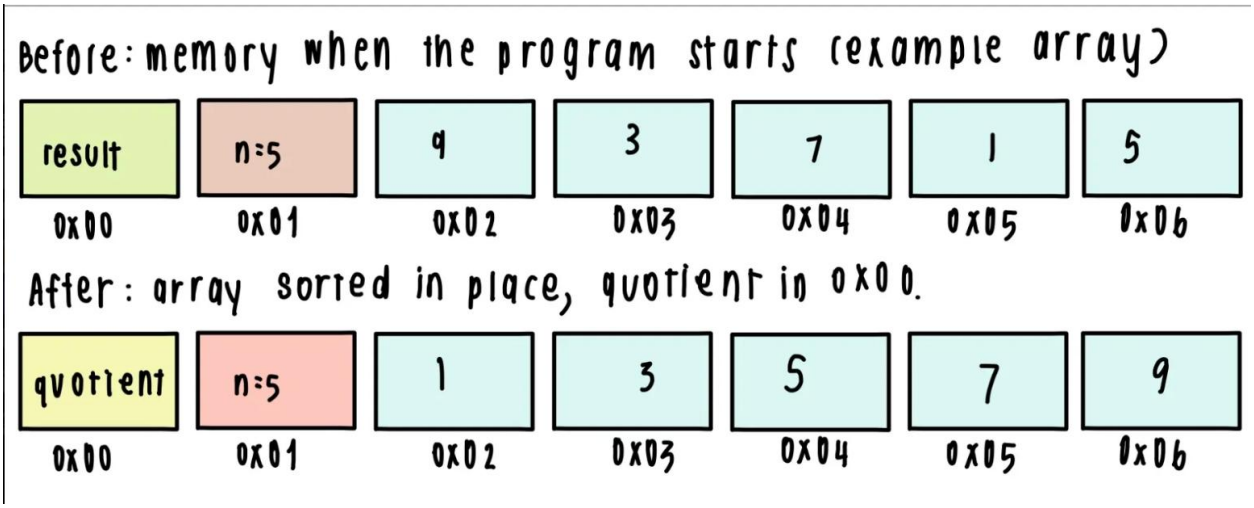
Reading	What it covers	Think about
Wikipedia: "Simple-As-Possible computer" (SAP-1)	A real tiny CPU: its parts, its instructions and its fetch and execute steps. Our CPU will look similar.	Which instruction would you add next?
Nand2Tetris (optional)	A free course where you build a simple computer from the ground up. Look at the "Hack" CPU.	How is it different from SAP-1?
Harris and Harris: Digital Design and Computer Architecture (optional)	A textbook. The chapters on architecture and microarchitecture cover the ISA and the datapath.	How do they split datapath and control?

7. Resources for the ISA challenge

Week 1 Teaser: your CPU must sort a randomly sizes array in place and write a quotient to address **0x00**. Your ISA has to make that possible. This section lists the ideas behind it. Each one has something short to watch or read.

This is not the final project, we will define it in the meeting on Saturday Nov 14.

The task in one picture



Division by repeated subtraction

Division always splits a number a into a quotient q and a remainder r:

$$a = q \cdot b + r, \qquad 0 \le r < b$$

With no divider, you can find q by subtracting b from a again and again. Count the subtractions. Stop when what is left is smaller than b. The count is q and what is left is r. In the slide task, the order of the two numbers and the case b = 0 are for your team to decide.

Ideas to learn, and where

Idea	Why the ISA needs it	Watch or read	Think about
Sorting in place	The array must end up sorted at the same addresses. No second copy.	**Video (search):** "Bubble-sort with Hungarian (Csángó) folk dance" (AlgoRythmics, about 4 min) **Read:** Bubble sort or Selection sort	Sorting is just "compare two numbers, swap if wrong". What instructions does that need?
Reaching array[i] when i changes	A loop walks along the array. The address changes each time, so it cannot be fixed in the instruction.	**Read:** Wikipedia: Addressing mode (look at "register indirect").	How can an instruction say "use the address held in a register"?
Divide with no divider	The ALU may not divide. Software must build division from subtract and compare.	**Read:** Wikipedia: Division algorithm (the "Slow division" part, and see the equation below).	How many times can b be subtracted from a before it goes below zero? What if b is 0?
Compare and jump	Loops and "swap if bigger" need a way to test a result and jump.	**Video:** Ben Eater 8-bit computer series on eater.net/8bit, the flags and conditional jump videos (search on his page). **Read:** Wikipedia: Status register	Which flags do you need: zero, carry, negative?
The stack (hint on the slide)	A stack gives you temporary storage with push and pop, so a few registers can go a long way.	**Video (search):** "Call Stacks - CS50 Shorts" **Video (search):** "What is a stack and how does it work? 6502 part 5" (Ben Eater) **Read:** Wikipedia: Stack	Could push and pop make swapping two array values easier? What register would hold the stack pointer?
Intel assembly syntax	The slides ask for Intel-style text, like mov dst, src. Ours will be a small cousin of it.	**Read:** x86 assembly quick guide (U. Virginia) **Read:** Wikipedia: x86 assembly (the "Syntax" part).	In "mov a, b", which one is the destination? Which Intel ideas do we copy, and which do we drop for 8 bits?

Before you design

- Write the program first in plain English: loop over the array, compare neighbours, swap, then divide. Every step you cannot do with your current instructions is a missing instruction.
- Keep it 8-bit. Only the PC may be wider, so addresses of 8 bits or fewer are the easy path.
- Single cycle means each instruction must finish in one clock tick. Ask of every new instruction: can the datapath do this in one step?

8. Words to know

Word	Simple meaning
Bit / byte	A bit is a 0 or a 1. A byte is 8 bits.
Instruction	One command for the CPU, stored as bits in memory.
ISA	The list of instructions a CPU understands, and how they are written in bits.
Opcode / operand	The opcode says which operation. The operand says which address or value to use.
PC (program counter)	A counter that holds the address of the next instruction.
IR (instruction register)	A register that holds the instruction being run right now.
Register	A small box (made of flip-flops) that remembers a number.
ALU	The part that does math and logic: add, subtract, AND, OR, shift.
Control unit	A state machine that tells every other part what to do on each clock.
Datapath	The parts that hold and move data: registers, ALU and the wires between them.
Flag	A 1-bit memory of a fact, such as "the last result was zero".
Clock	A signal that ticks. Registers change only on the ticks.
RTL	The Verilog description of the hardware: registers and the logic between them.

9. Questions to bring to the next session

Write down your answers, or your questions. It is fine to not know.

1. In your own words, what happens in fetch, decode and execute?
2. Which of our modules remembers something, and which one only calculates?
3. Pick one instruction from the example table. Which modules does it use, and in what order?
4. Think of a tiny program (for example "count to 3"). Which instructions would it need?

5. For the sorting step, write the compare-and-swap of two neighbours as plain-English steps. Which of them need a new instruction?
6. How will your program reach array[i] when i changes? Which instruction or register makes that possible?
7. Write division by repeated subtraction as steps. What does your ISA need to loop and to stop?
8. Where could a stack help you in this task, and which instructions would it add?
9. One thing from the videos or readings that you did not understand.